CAGT

Recall AI
Software Development Plan
Version 1.1

[Note: The following template is provided for use with the Rational Unified Process. Text enclosed in square brackets and displayed in blue italics (style=InfoBlue) is included to provide guidance to the author and should be deleted before publishing the document. A paragraph entered following this style will automatically be set to normal (style=Body Text).]

[To customize automatic fields in Microsoft Word (which display a gray background when selected), select File>Properties and replace the Title, Subject and Company fields with the appropriate information for this document. After closing the dialog, automatic fields may be updated throughout the document by selecting Edit>Select All (or Ctrl-A) and pressing F9, or simply click on the field and press F9. This must be done separately for Headers and Footers. Alt-F9 will toggle between displaying the field names and the field contents. See Word help for more information on working with fields.]

Revision History

Date	Version	Description	Author
21/Jun/26	1.0	Initial draft - project inception	Thái Nguyễn Tuấn Kiệt
08/Jun/26	1.1	Add Sections 1, 2, 3, 4.2, 4.3 - elaboration phase	Thái Nguyễn Tuấn Kiệt
12/Jul/26	1.2	Revised version for PA2. Detailed schedule and iteration plan for sprints 3, 4, 5. Updated risk management	Thai Nguyen Tuan Kiet

Table of Contents

Software Development Plan

1. Introduction

This Software Development Plan (SDP) describes the overall planning, organization, and management approach for the Recall AI project, developed by Group 07 (CAGT) as part of the Introduction to Software Engineering course.

Recall AI is a web-based proactive study assistant that helps university students turn raw study materials into a structured, verifiable knowledge-acquisition process. Unlike passive review tools (re-reading, highlighting), Recall AI uses an AI Examiner to conduct multi-turn oral-style quizzes strictly based on the user's uploaded material, and employs a Concept Graph Engine — a Directed Acyclic Graph (DAG) of prerequisite relationships — to automatically detect and remediate root-cause knowledge gaps through a depth-limited reverse BFS trace-back algorithm.

This document covers:

- Section 2: Project Overview — purpose, scope, assumptions, constraints, and deliverables.
- Section 3: Project Organization — team structure, roles, and responsibilities.
- Section 4: Management Process — estimates, phase/iteration plan, schedule, resourcing, reporting, risk management, and configuration management.

This plan is a living document and will be updated at the beginning of each sprint to reflect current team decisions and progress.

2. Project Overview

2.1 Project Purpose, Scope, and Objectives

Recall AI addresses three linked problems in student self-study:

- Passive learning, shallow review - re-reading and highlighting have low retention effectiveness compared to retrieval practice.
- No objective self-assessment - students cannot reliably distinguish genuine understanding from mere text familiarity.
- Unplanned review - without a structured plan, students cram before deadlines without prioritizing by concept difficulty or remaining time.

The project's goal is to build a working MVP web application that demonstrates the technical feasibility of the trace-back algorithm: when a student answers a concept incorrectly, the system traverses the prerequisite graph to identify and schedule review of the underlying foundational gap - rather than simply repeating the same question.

Scope (MVP - 5 sprints, 10 weeks)

In Scope	Out of Scope
User authentication (email + password)	Google OAuth / third-party SSO

AI Study Planner (text, image and PDF input)	DOCX direct file parsing
Concept Graph Engine (DAG, trace-back BFS)	Full probabilistic model (ALEKS-style)
Focus Session (Pomodoro timer, auto interruption detection)	Mobile app
AI Examiner (multi-turn interview, 3-turn limit/concept)	Voice input (Speech-to-Text)
Dashboard (concept graph colored by mastery score)	Social/collaborative features
Proactive reminders (urgency-score based)	Third-party calendar integration

Objectives

- Deliver a working MVP by Sprint 5 (Aug 23, 2026) that satisfies all 8 primary use cases (UC-01 to UC-08).
- Demonstrate the Concept Graph Engine with real trace-back behavior on at least one end-to-end demo scenario.
- Maintain a stable, testable codebase - the trace-back algorithm must be unit-testable with mock data, independent of AI quality.
- Complete all 5 PA assignments on time and with sufficient quality.

2.2 Assumptions and Constraints

Assumptions:

- All 5 team members are available for the full 10-week project duration (Jun 21 – Aug 23, 2026).
- The Gemini API free tier provides sufficient quota for development and demo use (≤ 50 sessions/day).
- Target users are first- and second-year university students studying subjects with layered concept structures (e.g., algorithms, networks, operating systems).
- Study materials submitted by users are readable plain text or clear photographs; heavily handwritten or low-resolution image inputs are out of scope for MVP.
- The Concept Graph Engine's trace-back quality depends on the AI correctly extracting prerequisite relationships; the mandatory human-in-the-loop confirmation step (UC-03) mitigates incorrect AI extractions.

Constraints:

- Fixed schedule - 10 weeks (5 × 2-week sprints). No schedule extension is possible due to course deadlines.
- Zero budget - all tools and services must use free tiers (Gemini API free tier, Vercel/Render free hosting, GitHub free, Figma free).
- Team size fixed at 5 - no additional members may be added.

- C4: AI must not be used as an orchestrator - all routing decisions (when to stop, when to switch concepts, when to trigger trace-back) are deterministic software logic, not AI decisions. AI is called only for generate_question and grade_answer with fixed JSON output schemas.
- C5: The AI Examiner must never generate questions outside the user's uploaded material - no external knowledge fabrication.
- C6: The maximum number of question-answer turns per concept is fixed at 3 turns to control API cost and response time.

2.3 Project Deliverables

A list of the artifacts to be created during the project, including target delivery dates.

Deliverable	Description	Target Date
PA0 - Project Proposal	Project proposal document, team info, interview evidence, tech stack	Jun 28, 2026
PA1 - Requirements Specification	Vision document, user personas, user stories (23 stories across 5 modules), problem statements, hypothesis statements	Jul 12, 2026
PA2 - Use Case Model & SDP	Use case diagram (Draw.io), 8 use case specifications (RUP format), Software Development Plan (this document)	Jul 12, 2026
PA3 - Working Software v1	Auth, Dashboard skeleton, AI Study Planner, Concept Graph Engine with mock data, Focus Session	Jul 26, 2026
PA4 - Working Software v2	AI Examiner (multi-turn state machine), Concept Graph Engine connected to real grading data, test conversation flows	Aug 09, 2026
PA5 - Final Product + Demo	Dashboard concept graph visualization, agentic reminder layer, full system test, final demo	Aug 23, 2026
Weekly Reports	Weekly status reports for every sprint week, tracking completed tasks, blockers, and planned work	Every Tuesday

3. Project Organization

3.1 Organizational Structure

The project team follows a flat, role-based Agile structure with a Project Manager coordinating sprint planning and reporting. There is no external reviewer - all review is done peer-to-peer within the team.

Role	Member	Responsibilities
Project Manager & UI/UX Designer	Thái Nguyễn Tuấn Kiệt	Sprint planning & tracking, JIRA board management, UX research, Figma wireframes & hi-fi mockups, weekly status reports, PA submission coordination
System Architect & Fullstack Dev	Nguyễn Thế Quân	System architecture design, database schema design (PostgreSQL/Prisma, concepts & concept_edges tables), Concept Graph Engine algorithm, API design & code review
QA / Tester & Fullstack Dev	Nguyễn Minh Phát	Requirements elicitation (SRS), use case model & specifications, test plan & test cases, functional testing, backend development support
Frontend Leader	Nguyễn Phương Gia Bảo	React/Vite app scaffold & routing, UI component library, authentication UI, Dashboard layout, concept graph visualization (react-flow), API integration
Backend Leader	Ngô Văn Phong	Express.js REST API implementation, authentication system (JWT + bcrypt), database models & migrations, Swagger/OpenAPI documentation, mock API responses

Communication channels:

- Discord - primary day-to-day communication (separate channels per role/sprint)
- JIRA - task tracking and sprint management
- GitHub - source code, pull requests, code review
- Google Drive - document storage and sharing
- Weekly Scrum Meeting - every Saturday, 30–60 minutes, full team

3.2 Roles and Responsibilities

RACI Matrix for Key Activities:

Activity	Kiệt (PM)	Quân (Arch)	Phát (QA)	Bảo (FE)	Phong (BE)
Sprint planning & tracking	R/A	C	I	I	I
Architecture design	C	R/A	I	C	C
UI/UX design (wireframes, mockups)	R/A	C	C	C	C
Concept Graph Engine design & impl.	C	R/A	C	I	C
Graph visualization (Dashboard)	I	C	I	R/A	C
Frontend development	I	C	I	R/A	I
Backend API development	I	C	I	I	R/A
AI integration (Gemini API)	I	R	I	C	A
Requirements & use case documentation	C	C	R/A	I	I
Testing	I	I	R/A	C	C
Reporting & documentation	R/A	C	C	I	I

R = Responsible, A = Accountable, C = Consulted, I = Informed

4. Management Process

4.1 Project Estimates

The project has a fixed schedule of 10 weeks across 5 two-week sprints. The team consists of 5 members with zero budget. All tooling uses free tiers. No re-estimation is expected unless a sprint rolls over more than 20% of its tasks, in which case the Project Manager will convene a planning session to adjust the next sprint's scope.

4.2 Project Plan

4.2.1 Phase and Iteration Plan

The project is planned over 10 weeks, divided into 5 iterations (Sprints) following a tailored RUP (Rational Unified Process) lifecycle.

- Phase 1: Inception
  - Iteration: Sprint 1 (Jun 21 – Jun 28, 2026)
    - Objectives: Form the team, select the project idea, set up development tools (GitHub, Jira, Slack), and finalize the Project Proposal (PA0).
- Phase 2: Elaboration
  - Iteration 1: Sprint 2 (Jun 29 – Jul 12, 2026)

- Objectives: Requirements elicitation and initial design. Complete PA1 and PA2. Specify Vision, Use Cases, System Architecture, Database schema, and design UI wireframes.

+ Iteration 2: Sprint 3 (Jul 13 – Jul 26, 2026)
  - Objectives: Detailed design and test planning (PA3). Setup core architecture (Frontend & Backend). Build the first working software (MVP Phase 1) including Authentication, AI Study Planner logic, and a mock version of the Concept Graph Engine.

- Phase 3: Construction
  + Iteration 1: Sprint 4 (Jul 27 – Aug 9, 2026)
    - Objectives: Implement the AI Examiner (multi-turn interview). Connect the grading results to the Concept Graph Engine built in Sprint 3. Test conversation flows. Deliver PA4.
  + Iteration 2: Sprint 5 (Aug 10 – Aug 23, 2026)
    - Objectives: Implement Concept Graph visualization (react-flow) on the Dashboard. Finalize the agentic trace-back algorithm. Conduct full system testing with real data, and prepare the final demo. Deliver PA5.

4.2.2 Releases

- *Release 0.1 (Internal Alpha) - Target: End of Sprint 3*
  + *Description: The first working skeleton of the application. It includes basic user authentication, the ability to upload materials and extract concepts (AI Study Planner), and a mock Concept Graph Engine. This is an internal release to verify architecture.*
- *Release 0.5 (Beta) - Target: End of Sprint 4*
  + *Description: This release integrates the AI Examiner. Users can conduct multi-turn oral quizzes, and the system can trace back weaknesses to foundational concepts. Focuses on testing the core logic without full visual polish.*
- *Release 1.0 (Final Demo) - Target: End of Sprint 5*
  + *Description: The complete MVP system. Includes the visual Dashboard (Concept Graph visualization) and all agentic features. Fully tested and ready for the final course presentation and grading.*

4.2.3 Project Schedule

Key Milestones:

Date	Milestone	Achievement Criteria
Jun 28, 2026	PA0 Submission	Project proposal + team setup complete
Jul 01, 2026	User interviews complete	≥ 5 interviews documented with evidence links
Jul 08, 2026	Use Case specifications done	8 UCs in RUP format, UC diagram in Draw.io
Jul 12, 2026	PA1 + PA2 Submission	Vision doc, SDP, Use Cases, wireframes submitted to Moodle
Jul 26, 2026	PA3 - Alpha release	Auth + Planner + Concept Graph Engine (mock) + Focus Session deployed to staging
Aug 09, 2026	PA4 - Beta release	AI Examiner + trace-back connected to real grading data, demo-ready
Aug 23, 2026	PA5 - Final Submission	Full MVP deployed, demo recorded/performed, all documentation complete

4.3 Project Monitoring and Control

4.3.1 Reporting

Weekly Status Reports:

- Published every Saturday at the end of each sprint week.
- Format: tasks completed this week, blockers encountered, tasks planned for next week, current sprint completion percentage.
- Stored in: /docs/management/weekly-reports/week-N.md on GitHub.

Sprint Review (end of each sprint):

- Full team retrospective meeting (30–60 min, recorded in Discord).
- Sprint review document: what was achieved, what was not, root causes, adjustments for next sprint.

JIRA Board:

- Tasks created for every deliverable before the sprint begins.
- Status columns: To Do → In Progress → In Review → Done.
- PM (Kiệt) reviews JIRA board every Monday and Friday.
- Burndown chart generated automatically to track sprint velocity.

Informal Communication:

- Daily check-ins in Discord #daily-standup channel (async text format: Done / Doing / Blocked).
- Blocker escalation: if a task is blocked for > 1 day, tag PM in #blockers channel.

4.3.2 Risk Management

Risk ID	Risk Description	Prob.	Impact	Exposure	Priority	Mitigation / Contingency Plan
R01	Gemini API rate limit exceeded during demo	3	5	15	High	Cache AI results; limit requests/user/day; implement fallback to static flashcard mode with pre-generated questions
R02	AI returns incorrect prerequisite relationships → poor trace-back quality	4	4	16	High	Mandatory human-in-the-loop confirmation (UC-03) before any study session; UI allows easy node/edge editing
R03	Trace-back algorithm produces a cyclic graph	3	4	12	High	DAG check (cycle detection) runs on every edge save; edges causing cycles are rejected with user notification and logged
R04	Workload imbalance - PM overloaded, Backend tasks not started	4	3	12	High	Explicit RACI per sprint; rollover tasks assigned with hard deadlines; PM task count reduced Sprint 2 (10→6)

R05	Concept Graph Engine not complete by Sprint 3 deadline	2	5	10	Medium	Algorithm unit-tested with mock data independently of AI; minimum demo criterion = graph saves + BFS runs on mock; Dashboard viz deferred to Sprint 5 if needed
R06	AI grading is inconsistent across turns	3	3	9	Medium	Grading rubric extracted from material once at Ingest; AI only grades against that fixed rubric per turn; users can dispute grading results
R07	Team member unavailable mid-sprint	2	4	8	Medium	Each technical component has a primary owner and one secondary (see RACI); critical path items have Quân as backup
R08	Study material with no layered concept structure (sparse graph)	3	2	6	Low	System applies ordinary spaced repetition for isolated concepts; trace-back gracefully skips concepts with no prerequisites
R09	Frontend–Backend interface mismatch	2	3	6	Low	Swagger/OpenAPI documentation due Aug 10; frontend uses mock data until backend API is stable; contract-first API design
R10	Gemini API pricing changes / free tier removed	1	5	5	Low	AI service abstraction layer designed for provider switching; alternative: OpenAI API with identical JSON schema
R06	React-flow visualization learning curve for Frontend team.	2	3	6	Medium	Start with basic nodes/edges coloring APIs; refer to react-flow example templates.
R07	AI grading inconsistency on free-form text answers.	3	4	12	High	Extract grading rubrics once during document ingestion; provide grade dispute interface.
R08	Excessive API usage costs from multi-turn chats.	3	3	9	Medium	Enforce strict limit of 3 turns per concept and daily limits per user.
R09	Concept graph contains cycles (violating DAG constraint).	2	5	10	High	Implement topological sort cycle check on every save, automatically alert and reject cycle edge.

4.3.3 Configuration Management

Document Management:

- Platform: GitHub (https://github.com/Lade1q/planning-ai/)

- /docs/requirements/ - Vision doc, SRS, personas, user stories, interview findings
- /docs/analysis-and-design/ - SAD, DB schema, wireframes, mockups, use case specs
- /docs/management/ - SDP, weekly reports, sprint reviews
- /pa/pa0/, /pa/pa1/, /pa/pa2/, ... - PA submission packages
- All final submission packages are zipped as PA[N]-Group07.zip and uploaded to Moodle.

Code Quality Tools:

- ESLint + Prettier - configured at project init for both frontend and backend.
- TypeScript - strict mode enabled; types/interfaces shared between frontend and backend layers.
- Zod - runtime validation for all API request/response payloads.
- Jest - unit tests for the trace-back algorithm and scheduling logic (no AI dependency required).

Environment Management:

- .env.example committed to repo; actual .env files gitignored.
- Environment variables: DATABASE_URL, JWT_SECRET, JWT_REFRESH_SECRET, GEMINI_API_KEY, NODE_ENV.
- Three environments: development (local), staging (Vercel/Render preview), production (Vercel/Render main).